HEALTHCARE DATA ANALYTICS INTERNSHIP · PROJECT REVIEW

Project Review: Process, Challenges and Roadmap

Development process and tech stack · challenges and solutions · future enhancements · performance · scalability · implementation efficiency

Project: 30-day readmission and resource use among diabetic inpatients (UCI Diabetes 130-US Hospitals, 1999–2008)

Prepared by: Juluru Akshay
Repository: https://github.com/AJ1206/healthcare-readmission-analytics
Date: 24 October 2026

Contents

Summary 3

1. Step-by-Step Development Process 4

1.1 Tech stack . . . 4

2. Challenges and How They Were (or Could Be) Solved 6

2.1 Data challenges . . . 6

2.2 Methodological challenges . . . 6

2.3 Engineering and tooling challenges . . . 6

3. Future Enhancements and Upgrades 8

4. Optimising Performance 9

4.1 Optimisations ranked by expected impact . . . 9

4.2 Model inference performance . . . 9

5. Making the Project Scalable 10

5.1 Governance at scale . . . 10

6. Improving Implementation Efficiency 11

6.1 Example: development sample . . . 11

7. Prioritised Improvement Roadmap 11

Summary

The project is a reproducible, end-to-end healthcare analytics pipeline. It downloads a public dataset of 101,766 inpatient encounters, validates and cleans it, builds an analysis cohort of 69,970 patients, runs statistical tests and a predictive model (random forest, test ROC-AUC 0.649), and produces an interactive dashboard, a Power BI build pack, a 61-page integrated report and six weekly reports, all from one command in about nine minutes.

This review explains how the project was built, the problems met along the way and how they were solved, and where it can go next. The biggest single opportunity is clear from the measurements: predictive modelling takes about three-quarters of the run time, and most of that is random-forest tuning.

Topic	Key takeaway
Process	Eight scripted steps, each with fixed inputs and outputs; every number traceable to code
Challenges	Data gaps, repeated patients, leakage risk, modest model performance and several tooling problems — all resolved or documented
Enhancements	Current local data, explainability (SHAP), recalibration, survival models, EHR integration
Performance	Faster tuning (successive halving), Parquet storage, caching unchanged steps
Scalability	Columnar storage and DuckDB/Spark for volume; orchestration, containers and a model registry for operations
Efficiency	Config files, tests, CI, a small development sample and automated document builds

1. Step-by-Step Development Process

The project followed a CRISP-DM-style lifecycle with added quality, ethics and communication stages. Each stage became a numbered Python script so it could be re-run and audited.

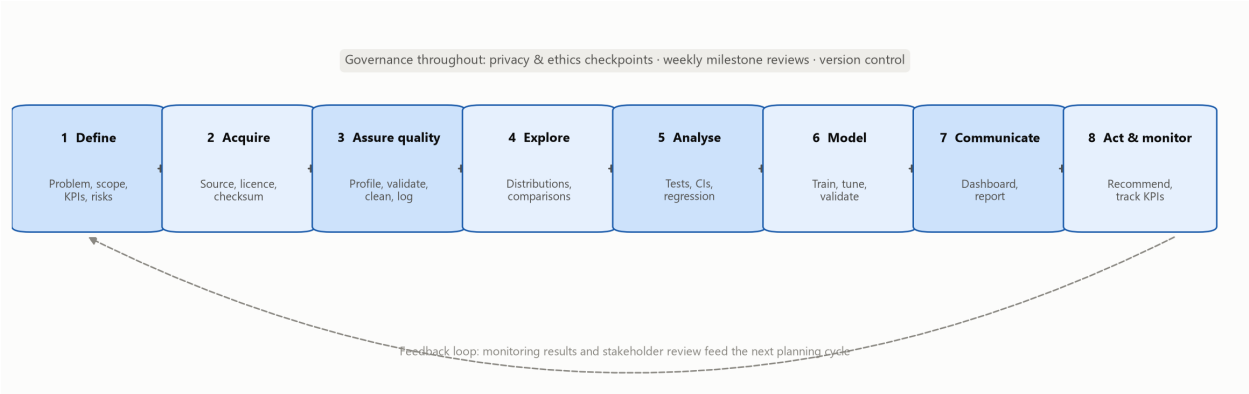


Figure 1. Project workflow (adapted from CRISP-DM).

#	Step	What was done	Output
1	Define the problem	Healthcare question, objectives, scope, KPIs, hypotheses, risks and a six-week roadmap	Strategic plan
2	Research data sources	Compared seven sources on nine weighted criteria; verified access programmatically	Dataset choice (UCI, score 4.50/5)
3	Set up the environment	Isolated Python virtual environment; pinned package versions	requirements.txt
4	Acquire data	Scripted download; SHA-256 checksum; provenance and licence recorded; raw data never edited	01_data_acquisition.py
5	Profile the data	Types, missing values ('?' markers), duplicates, invalid codes, distributions	Profiling tables
6	Validate and clean	44 rules (range, domain, cross-field, uniqueness); documented cleaning; issue log	02_data_quality_and_cleaning.py
7	Define the cohort	First encounter per patient; deaths and hospice excluded	69,970-patient cohort
8	Explore	Descriptive statistics and 12 figures with confidence intervals	03_exploratory_data_analysis.py
9	Test hypotheses	Chi-square, Mann–Whitney, Kruskal–Wallis, Holm correction, logistic regression	04_statistical_analysis.py
10	Model	Baselines + 3 models; 60/20/20 patient split; CV tuning; one test evaluation; calibration; fairness	05_predictive_modeling.py
11	Visualise	Executive KPI figure, offline HTML dashboard, Power BI star schema and DAX	06_visualization_dashboard.py
12	Report	Word reports generated from outputs; references checked against Crossref	07 and 08 scripts
13	Quality-check documents	Word refreshes contents pages; PDFs rendered and inspected page by page	Verified reports
14	Prove reproducibility	Deleted all outputs and re-ran from the raw download: identical results	Reproducibility check
15	Package and publish	Full and GitHub ZIPs; public GitHub repo with MIT licence; demo script	GitHub repository

1.1 Tech stack

Layer	Tools (version used)	Purpose
Language	Python 3.14.3	All processing, analysis and document generation

Layer	Tools (version used)	Purpose
Data handling	pandas 3.0.6, NumPy 2.5.4	Loading, profiling, cleaning, reshaping
Statistics	SciPy 1.18.1, statsmodels 0.15.0	Hypothesis tests, confidence intervals, logistic regression, VIF
Machine learning	scikit-learn 1.9.1, joblib 1.6.0	Pipelines, tuning, models, metrics, model saving
Visualisation	Matplotlib 3.11.2, seaborn 0.13.2	Publication-quality static charts (colour-blind-safe palette)
Dashboard	HTML, CSS, JavaScript (no libraries); Power BI + DAX	Offline interactive dashboard; Power BI build pack
Documents	python-docx 1.2.0; Microsoft Word (COM via PowerShell); ReportLab	Word reports, TOC refresh, this PDF
Quality checks	PyMuPDF; headless Microsoft Edge	Render pages for inspection; dashboard screenshots
Data access	requests 2.34.2; Crossref API	Download data; verify references
Version control	Git 2.55, GitHub, GitHub CLI	History, publishing, collaboration
Platform	Windows 11, PowerShell	Development and execution

2. Challenges and How They Were (or Could Be) Solved

The challenges below were all encountered in this project. Each row gives why it mattered and the solution used, plus a better long-term option where one exists.

2.1 Data challenges

Challenge	Why it mattered	Solution
Best datasets were restricted (MIMIC-IV, HCUP)	Credentialing or purchase would take weeks	Weighted selection favoured an open CC BY 4.0 dataset; restricted sources kept as future options
Missing values coded as '?' and 'NULL/Not Mapped'	Standard tools would not count them as missing	Converted markers to true missing values; mapped codes with the official mapping file
Weight 97% missing; specialty 49%; payer 40%	Imputation would fabricate data	Dropped weight; kept others with an explicit 'Unknown' level; documented the effect
Completeness metric initially overstated	Counting 'Unknown' as complete inflated the improvement	Redefined the metric to treat 'Unknown' as missing (honest 95.8% → 97.6%)
Repeated encounters (up to 40 per patient)	Breaks independence; leaks patients across train/test	One index encounter per patient; grouped CV suggested as a sensitivity analysis
Data from 1999–2008, no dates or hospital IDs	No trends, benchmarking or temporal validation	Stated as a limitation; future work uses current local data

2.2 Methodological challenges

Challenge	Why it mattered	Solution
Risk of target leakage	Would inflate model performance	Prediction-time rule; outcome fields excluded; preprocessing inside pipelines
Rare outcome (9%)	Accuracy is misleading (91% by predicting 'no')	PR-AUC, sensitivity and PPV; threshold chosen from workload (top 20%)
Large sample makes everything 'significant'	p-values alone mislead	Effect sizes, confidence intervals and Holm correction
Modest model performance (AUC 0.649)	Not accurate enough for individual decisions	Honest reporting; recommendations rely on statistical findings; richer data proposed
Calibration slope 1.26	Risk estimates compressed toward the mean	Reported; Platt/isotonic recalibration proposed before any use
Baseline flagged every patient	Tied scores at the cut-off	Strict threshold ($p > t$) made it the intended 'any prior admission' rule
Fairness concerns	Race and payer could encode inequity	Excluded as predictors; subgroup audit instead

2.3 Engineering and tooling challenges

Challenge	Solution used	Better long-term option
Python not on PATH; stale Anaconda entry	Located the installed interpreter with the py launcher	Document setup in README; use uv or conda environment files
Virtual environment failed on a very long path	Created it at a short path	Keep projects in short paths (e.g. C:\dev\project)
Power BI Desktop cannot be automated	Offline HTML dashboard + Power BI build pack	Power BI service with a scheduled dataset refresh
Word tables of contents do not update by themselves	Refresh via Word automation (PowerShell COM)	LibreOffice headless or docx2pdf in CI
Hidden Word dialogs froze automation	Closed orphaned processes; verified documents from file contents	Time-outs and a clean Word profile for automation

Challenge	Solution used	Better long-term option
Headless Edge screenshots silently failed	Separate browser profile and temp copy of the page	Playwright for reliable browser automation
Unicode errors in the Windows console	PYTHONIOENCODING=utf-8	Set UTF-8 mode globally (PYTHONUTF8=1)
Project files lost during a storage change	Regenerated everything from the scripted pipeline; checksums matched	Commit early to GitHub; keep work outside temporary folders
GitHub showed the licence as 'Other'	Standard MIT text in LICENSE; data licence in README	Use GitHub's licence templates

3. Future Enhancements and Upgrades

Area	Enhancement	Benefit	Effort
Data	Current local hospital data (ICD-10) with admission dates and hospital IDs	Relevance; trends; temporal validation; benchmarking	High
Data	Add vital signs, lab values, social determinants, follow-up visits	Better prediction (current AUC is modest)	High
Data	Link CMS HRRP hospital-level data as a separate benchmark study	Hospital comparisons	Medium
Analytics	Time-series forecasting of monthly readmissions (ETS/ARIMA)	Capacity planning	Medium
Analytics	Survival models (time to readmission, death as competing risk)	Uses timing information	Medium
Analytics	Propensity-score analysis of HbA1c testing	Stronger evidence than plain association	Medium
Model	SHAP explanations per patient	Transparent, clinician-friendly reasons	Low
Model	Recalibration (isotonic/Platt) and decision-curve analysis	Trustworthy risks; shows clinical usefulness	Low
Model	Fairness mitigation and monitoring dashboards	Equitable performance across groups	Medium
Model	NLP on discharge summaries	Captures information missing from coded data	High
Product	Build the Power BI .pbix and publish to the Power BI service	Organisation-wide access with security	Low
Product	Streamlit app for what-if exploration and patient scoring	Interactive model demonstration	Low
Integration	FHIR-based worklist inside the EHR	Risk flags where clinicians work	High
Evaluation	Stepped-wedge pilot of the discharge interventions	Measures real impact on readmissions	High
Reporting	Automated plain-language summaries for each refresh	Faster communication to non-technical staff	Low

4. Optimising Performance

Optimisation should start from measurement. The full pipeline takes about nine minutes; predictive modelling accounts for 74% of that, and random-forest tuning alone for 40%.

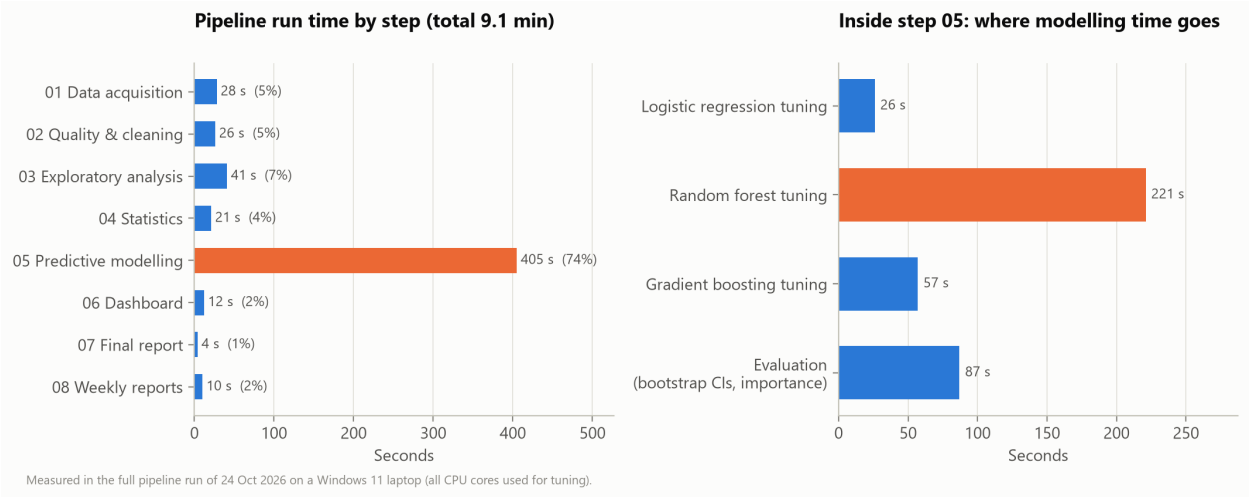


Figure 2. Measured run time per step and inside the modelling step.

4.1 Optimisations ranked by expected impact

Optimisation	Where	Expected effect
Successive-halving search (HalvingRandomSearchCV) instead of full randomised search	Step 05	Large cut in tuning time; poor candidates are dropped early on small samples
Fewer trees / depth limits for the random forest, or select gradient boosting	Step 05	Faster training and a much smaller model file (currently about 20 MB); gradient boosting's test AUC was only about 0.002 lower
Fewer bootstrap resamples and permutation repeats (or run them on a sample)	Step 05	The 87 s evaluation phase is dominated by bootstrap confidence intervals and permutation importance
Store cleaned data as Parquet with categorical types	Steps 02–08	Smaller files (38 MB CSV) and faster loading in every later step
Cache: skip a step when its inputs have not changed (Make, DVC)	All	Re-runs after small edits take seconds instead of minutes
Remove slow access probes from routine runs	Step 01	About 20 s of the 28 s is spent checking alternative datasets; the download itself takes about 4 s
Vectorise the ICD-9 grouping (numeric ranges with pd.cut)	Step 02	Replaces a Python function applied to 300,000 codes
Pre-aggregate dashboard data instead of embedding row-level data	Step 06	Smaller, faster dashboard (currently 1.9 MB)
Run independent steps in parallel (statistics and EDA)	03–04	Shorter wall-clock time

4.2 Model inference performance

- Batch scoring of all discharges once a day is cheaper than scoring on every request.
- A smaller model (gradient boosting or a pruned forest) loads and scores faster in an API.
- Export to ONNX for fast, language-independent inference if real-time scoring is needed.

5. Making the Project Scalable

The current design runs comfortably on one laptop for about 100,000 records. Scaling has two dimensions: more data (more hospitals, more years, more variables) and more use (more users, regular refreshes, live scoring).

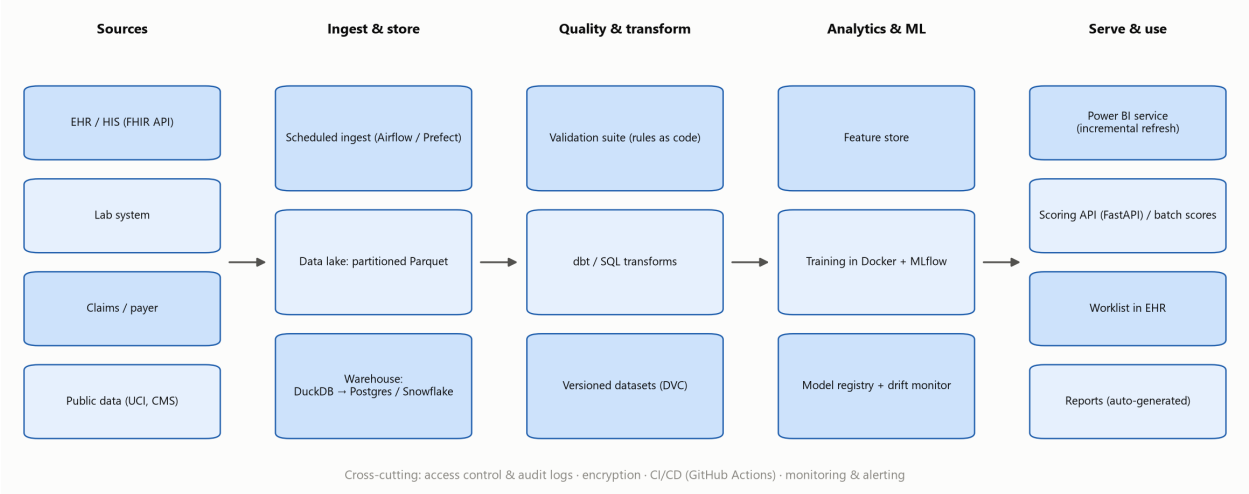


Figure 3. Target architecture for a multi-hospital, regularly refreshed deployment.

Dimension	Current	Scalable approach
Data volume	CSV files in memory with pandas	Partitioned Parquet; DuckDB for single-machine analytics; Spark or Polars for very large data
Storage	Local folders	Data lake + warehouse (PostgreSQL, Snowflake or BigQuery) with access control
Ingestion	Download from UCI	Scheduled, incremental loads from EHR (FHIR), lab and claims systems
Transformations	pandas scripts	SQL/dbt models for shared logic; validation suite run on every load
Orchestration	run_pipeline.py	Airflow or Prefect with retries, schedules and alerts
Environment	Local virtual environment	Docker images for identical runs anywhere
Modelling	Single training run	MLflow tracking and model registry; scheduled retraining; drift monitoring
Serving	Saved model + demo script	FastAPI scoring service or nightly batch scores written to the warehouse
Dashboards	Static HTML file	Power BI service with incremental refresh and row-level security per hospital
Configuration	Constants in code	YAML configuration per site (paths, thresholds, code mappings)
Multi-site	One dataset	Hospital ID in every table; per-site validation and calibration

5.1 Governance at scale

- Role-based access and audit logs for all identifiable data (DPDP Act 2023 and Rules 2025 in India).
- Data-sharing agreements and a data-protection impact assessment for every new site.
- A model card and monitoring report for every model version.

6. Improving Implementation Efficiency

Strategy	How	Benefit
Work on a development sample	A 5–10% stratified sample for coding and debugging; full data only for final runs	Seconds per iteration instead of minutes
Configuration file	Move paths, seeds, thresholds and model grids to a YAML file	Change behaviour without editing code
Automated tests	pytest for cleaning rules, cohort logic and metric functions	Catches errors like the baseline threshold bug immediately
Continuous integration	GitHub Actions runs tests and the pipeline on a sample at every push	Broken changes never reach the main branch
Linting and formatting	ruff + pre-commit hooks	Consistent, readable code with less review effort
Incremental builds	Make or DVC so only changed steps re-run	Faster feedback loop
Shared helpers	Common modules (data loading, styles, document helpers) — already used	Less duplication; one place to fix
Documents as code	Reports generated from results — already used	Numbers can never drift from the analysis
Notebooks for exploration only	Explore in Jupyter, then move stable code into scripts	Speed of exploration with reliable pipelines
Logging and timing	Structured logs with durations per step	Bottlenecks are visible (as in Figure 2)
Commit early and often	Small Git commits with clear messages; push to GitHub daily	History and backup (avoids the file-loss problem)
Reproducible environments	Pin versions; lock files; Docker	No 'works on my machine' problems

6.1 Example: development sample

```
cohort = pd.read_parquet('data/processed/cohort.parquet')
if config['dev_mode']:
    cohort = cohort.groupby('readmit_30', group_keys=False).sample(frac=0.1, random_state=42)
```

7. Prioritised Improvement Roadmap

Horizon	Actions
Quick wins (1–2 weeks)	Successive-halving tuning; Parquet storage; drop probes from routine runs; pytest suite; GitHub Actions; YAML config; recalibration and SHAP; build the Power BI .pbix
Medium term (1–3 months)	DuckDB warehouse; Docker; MLflow; Prefect orchestration; decision-curve analysis; forecasting; Streamlit demo
Long term (3–12 months)	Local current data under a data-sharing agreement; FHIR integration; multi-site deployment with monitoring; pilot evaluation of interventions

These steps keep the project's main strengths — reproducibility, transparency and honest evaluation — while making it faster to work on, cheaper to run and ready for real organisational use.